AI Practice Problems

Section 1: Constraint Satisfaction Problems

Problem 1

Consider a CSP with variables W, X, Y, Z each with domain {1, 2, 3, 4, 5} and the following constraints:

- $W < X$
- $X = Y + 1$
- $Z = W + Y$
- $W \neq 3$
- Y must be odd

a) Identify all unary and binary constraints.

b) Draw the constraint graph.

c) Apply node consistency to remove invalid values from domains.

d) Apply arc consistency and show the resulting domains for each variable.

Problem 2

Using the map coloring problem for Australia (as shown in the lecture slides), with variables WA, NT, Q, NSW, V, SA, T and domains consisting of {Red, Green, Blue}:

a) If WA is assigned Red, apply forward checking and show the resulting domains.

b) After applying forward checking in part (a), which variable would you select next according to the Minimum Remaining Values (MRV) heuristic? Break ties using the degree heuristic. c) Apply the Least Constraining Value heuristic to select a value for the variable identified in part (b).

Problem 3

Consider a scheduling CSP with five tasks (A, B, C, D, E) that need to be assigned to three time slots (1, 2, 3). The constraints are:

- Tasks A and B cannot be scheduled in the same time slot
- Task C must be scheduled in a later time slot than Task A
- Tasks D and E must be scheduled in consecutive time slots
- Task B cannot be scheduled in time slot 3

a) Draw the constraint graph for this problem.

b) Apply node consistency to determine the domain of each variable.

c) Assign Task A to time slot 1 and apply forward checking to update the domains.

d) After the assignment and forward checking in part c, which variable would you choose next according to the MRV heuristic? Break ties using the degree heuristic.

Constraint Satisfaction Problems Analysis

I'll provide solutions to these problems and then offer additional problems of similar or increased difficulty.

Problem 4

a) Identifying Constraints

Unary Constraints:

- $W \neq 3$
- Y must be odd

Binary Constraints:

- $W < X$
- $X = Y + 1$
- $Z = W + Y$

b) Constraint Graph

The constraint graph would have nodes W, X, Y, and Z with the following edges:

- Edge between W and X ($W < X$)
- Edge between X and Y ($X = Y + 1$)
- Edge between W and Z ($Z = W + Y$)
- Edge between Y and Z ($Z = W + Y$)

c) Node Consistency

Applying node consistency for unary constraints:

- For W: Remove 3 from domain → $W \in \{1, 2, 4, 5\}$
- For Y: Keep only odd values → $Y \in \{1, 3, 5\}$
- X and Z have no unary constraints → $X, Z \in \{1, 2, 3, 4, 5\}$

d) Arc Consistency

Examining each binary constraint:

For $W < X$:

- For W = 1, X can be 2, 3, 4, 5
- For W = 2, X can be 3, 4, 5
- For W = 4, X can be 5
- For W = 5, X has no valid values → remove 5 from W's domain

For $X = Y + 1$:

- For Y = 1, X must be 2
- For Y = 3, X must be 4
- For Y = 5, X must be 6 (not in domain) → remove 5 from Y's domain

For Z = W + Y:

- For various combinations of W and Y, Z must equal their sum

After propagating all constraints:

- W ∈ {1, 2, 4}
- Y ∈ {1, 3}
- X ∈ {2, 4}
- Z ∈ {2, 3, 4, 5, 7}

Problem 5

Consider a university course scheduling problem with courses C1, C2, C3, C4, and C5, which must be assigned to rooms R1, R2, and R3, and time slots T1, T2, and T3. The constraints are:

- C1 and C3 must be taught by the same professor, so they cannot be scheduled at the same time
- C2 requires special equipment only available in R1
- C4 and C5 are large classes that can only fit in R3
- C1 must be scheduled before C2 (not necessarily immediately before)
- C3 and C4 must be at different times and in different rooms
- R2 is not available during time slot T3

a) Formulate this as a CSP by defining variables, domains, and constraints. b) Draw the constraint graph. c) Apply node consistency to determine the domain of each variable. d) Assign C1 to (R2, T1) and apply forward checking. e) After the assignment and forward checking, which variable would you choose next according to MRV? Break ties using the degree heuristic.

Problem 6

Consider placing five components (A, B, C, D, E) on a circuit board grid with positions labeled as (row, column) where rows and columns range from 1 to 4. The constraints are:

- No two components can occupy the same position

- Component A must be in row 1
- Component B must be adjacent (horizontally or vertically) to component C
- Components D and E must be at least 2 grid units apart (Manhattan distance)
- Component C cannot be placed in column 4
- Component D must be in a corner position

a) Define the variables, domains, and constraints for this CSP. b) Draw the constraint graph. c) Apply node consistency. d) If A is placed at position (1,2), apply forward checking and show the updated domains. e) After placing A at (1,2), use the MRV heuristic to select the next variable. Break ties with the degree heuristic. f) Apply the Least Constraining Value heuristic to select a value for the variable identified in part (e).

Problem 7

Consider a 4×4 Sudoku puzzle where each row, column, and 2×2 block must contain digits 1-4 without repetition. Additionally:

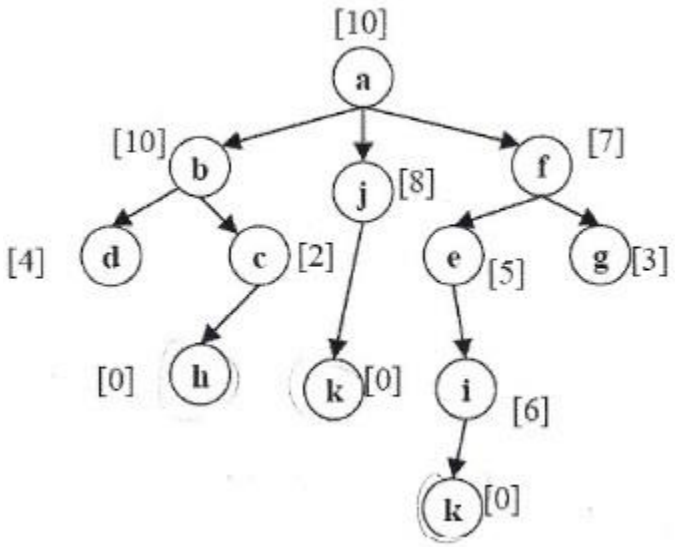
- The sum of digits in the first row must be even
- The sum of digits in the main diagonal must be greater than 8
- Cells (1,2) and (3,4) must have the same value
- The product of digits in the second column must be a multiple of 3

a) Formulate this as a CSP by defining variables, domains, and constraints. b) Identify all unary, binary, and global constraints. c) Apply node consistency for any unary constraints. d) If cell (1,1) is assigned the value 3, apply forward checking and show the resulting domains. e) After the assignment in part (d), which cell would you assign next according to MRV? Break ties using the degree heuristic. f) Use AC-3 algorithm to enforce arc consistency after the assignment in part (d). Show the queue of arcs and how domains change at each step.

Would you like me to elaborate on any of these problems or provide solutions?

Section 2: Local Searches

Problem 1



The provided graph represents a network of connected nodes with associated costs (shown in square brackets). Each node represents a location or state, and the directional edges represent possible transitions between states. The numbers in square brackets indicate either:

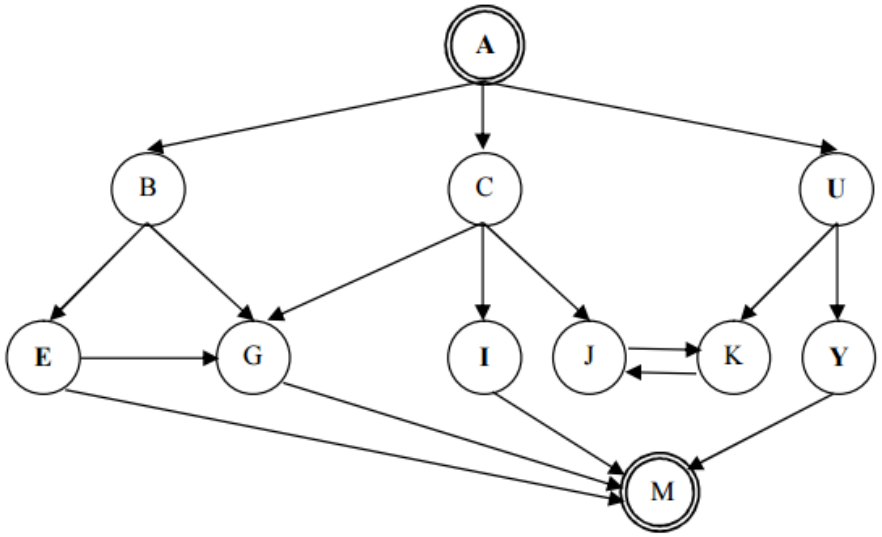
- For nodes: A heuristic value or intrinsic cost associated with that node
- For edges (implicit): The cost of traversing from one node to another

Apply hill climbing, simulated annealing, and genetic algorithms to find a path from node 'a' to any leaf node (nodes with no outgoing edges).

For each algorithm, run multiple trials and report:

- The best path found
- The associated total cost
- The number of iterations/generations required
- The consistency of solutions across runs

Problem 2



h(n):

h(A) = 7
h(B) = 5
h(C) = 3
h(U) = 4
h(E) = 2
h(G) = 3
h(I) = 6
h(J) = 2
h(K) = 1
h(Y) = 2
h(M) = 0

You are provided with a directed graph representing a state space for pathfinding. Each node in the graph has an associated heuristic value h(n) as listed on the right side of the image. Your task is to apply the A* search algorithm to find the optimal path from the start node A to the goal node M.

- The graph consists of 12 nodes: A, B, C, U, E, G, I, J, K, Y, and M, with directed edges connecting them as shown in the diagram.
- Node A is the designated start node (indicated by the double circle).
- Node M is the designated goal node (indicated by the double circle).
- Each node has an associated heuristic value h(n) that estimates the cost to reach the goal from that node:

Implement and compare the performance of multiple local search algorithms on this network:

- Hill Climbing
- Simulated Annealing

For each algorithm implementation:

- Define a suitable state representation for paths through the network

Analyze the algorithms' effectiveness in terms of:

- Solution quality (path cost/length)
- Computational efficiency (number of iterations or evaluations)

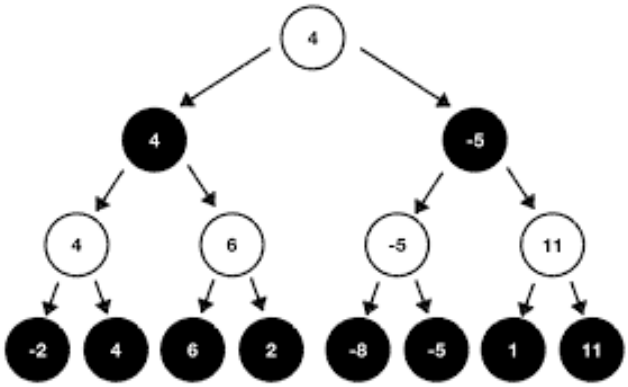
- Consistency across multiple runs
- Ability to escape local optima

Implement two variations of the hill climbing algorithm:

- Standard hill climbing
- Hill climbing with random restarts

Section 3: Adversarial Search Problems

Problem 1



The image depicts a game tree for a two-player, zero-sum game. Players alternate turns, with one player trying to maximize their score and the other trying to minimize it. The tree shows possible game states and terminal values, with black nodes representing maximizing player positions and white nodes representing minimizing player positions. The numbers in the terminal nodes represent the utility values at those end states.

Apply the minimax algorithm to determine the optimal strategy for both players, assuming perfect play.

For each non-terminal node in the tree:

- Calculate and show the minimax value
- Identify the optimal move for the player whose turn it is at that position

Implement the following variations of the minimax algorithm:

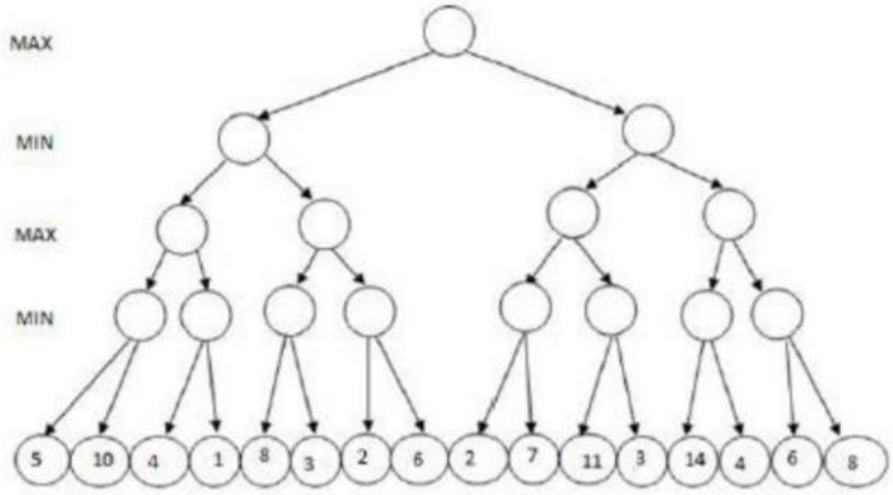
- Standard minimax without pruning

- Minimax with alpha-beta pruning

For the alpha-beta pruning implementation:

- Track the changes in alpha and beta values at each step
- Identify which branches can be pruned
- Compare the number of nodes evaluated with and without pruning

Problem 2



You are provided with a game tree representing a two-player, zero-sum game. The tree has 5 levels (including the leaf nodes) with MAX player's turns at levels 1 and 3, and MIN player's turns at levels 2 and 4. The leaf nodes contain utility values ranging from 1 to 14.

Your task is to apply the Alpha-Beta Pruning algorithm to efficiently compute the minimax value at each node of the tree.

Apply the standard Minimax algorithm to compute the value at each node, working from the leaf nodes upward. This will serve as a reference for the complete solution.

Implement Alpha-Beta Pruning to compute the same minimax values while potentially pruning branches that cannot affect the final decision. You must:

- Visit children from left to right as specified in the problem

- Initialize alpha = -∞ and beta = +∞ at the root node
- Track and update alpha and beta values at each node according to the algorithm rules:
    - For MAX nodes: Update alpha = max(alpha, child value)
    - For MIN nodes: Update beta = min(beta, child value)
- Prune a branch when alpha ≥ beta

For each node in the tree, document:

- The minimax value determined
- The alpha and beta values after processing the node
- Whether the node enabled any pruning of its siblings
- The specific comparison that led to pruning (if applicable)